

PARTE B: PYTHON AD ALTE PRESTAZIONI

SEZIONE 0. CHIAMARE C DA PYTHON

Chiamare C da Python

X

Chiamare codici scritti in C da Python: motivazioni ed esempi attuali. Primo aspetto da gestire: tipi di dati, libreria `ctypes`. Esempi. Osservazione: parallelizzazione di codici Python. Secondo aspetto da gestire: i puntatori. Terzo aspetto da gestire: le strutture C.

X

0. Voci correlate

- Introduzione alla Programmazione Avanzata e Parallela

1. Motivazioni ed Esempi

Motivazione. Python è un linguaggio *interpretato*, che è utile per creare delle interfacce in un maniera intuitiva e veloce, ma il suo tempo di esecuzione è molto più lento di un linguaggio compilato. Infatti, Python è noto di essere una *"glue language"*.

Da questo sorge la necessità di velocizzare dei codici scritti in Python, delegando alcune chiamate di funzioni ad altri linguaggi di programmazione più veloci, come C.

Esempio. NumPy e PyTorch fanno uso di implementazioni in C/C++.

Riusciremo a fare tutto questo con le *librerie condivise*, stando attenti ai seguenti aspetti:

- I tipi di dati degli argomenti e del valore di ritorno
- Gestione della memoria, assicurarsi che non siano dei "memory leak"
- Gestire alcune funzionalità che ci sono in C ma non in Python (o viceversa), come i puntatori o le strutture

X

2. Gestire Tipi di Dati

Problema. Molti tipi di dati tra C e Python sono rappresentati in una maniera profondamente diversa, tra cui:

- Gli interi hanno una precisione arbitraria in Python, invece in C è fissato (32/64 bit)

- I caratteri in C sono dei byte che codificano le lettere mediante la convenzione ASCII, invece in Python i caratteri sono dei simboli in Unicode (e quindi hanno più di un byte)

Per gestire la discrepanza tra i tipi di dati e quindi per cercare di convertirli, si usa la libreria standard `ctypes` di Python. In particolare, possiamo:

- Accedere alle librerie condivise scritte in C
 - Lo si fa scrivendo `code = ctypes.loadlib("...")`, con `code` possiamo effettuare le chiamate di funzioni in C
- Effettuare la conversione dei tipi di dati
- Specificare il tipo degli argomenti e del ritorno delle funzioni C
 - Lo si fa con gli attributi `code.fun.argtypes = ...` e `code.fun.restype = ...`

Osservazione. Un caso particolare sono i *puntatori*, infatti non esistono in Python! Per ottenere un puntatore ad un oggetto si chiama la funzione `pointer()` ad un oggetto con tipo fornito da `ctypes`. Per dereferenziare un puntatore, si chiama l'attributo `.contents`

Con questo, si può ad esempio scambiare due numeri in Python mediante i puntatori.

Adesso vediamo le strutture.

Supponiamo di definire una struttura `struct` che verrà utilizzata in C, e uno script Python vuole fare una chiamata al codice C. Per farlo, bisogna ridefinire la struttura con una *classe* in Python nel seguente modo:

```
class MYSTRUCT(ctypes.Structure):  
    _fields_ = [  
        ("f1", c_int),  
        ...  
    ]  
  
p = MYSTRUCT(...)
```

PYTHON

Ovviamente le strutture e le classi devono essere *sincrone*, prestando particolare attenzione anche al loro ordine.

Osservazione. Possiamo scaricare le librerie standard C, quindi è possibile chiamare le funzioni `malloc`, `printf`, `sizeof`, ... in Python! Inoltre, se il codice C è scritto con OpenMP, allora il codice Python che chiama quel codice diventa anch'esso parallelo. Ciò è molto utile, in quanto il tipo di parallelismo in Python è piuttosto limitato

SEZIONE A. OOP

Concetti Fondamentali dell'OOP

X

OOP: Cosa è, concetto di classe, attributo e metodo. Accesso agli oggetti: pubblico, protetto e privato. Concetto di ereditarietà tra classi.

X

1. Idea dell'OOP

Idea. Il concetto di lampadina ha alcune caratteristiche, tra cui *alcune cose che si possono fare* (accendere o spegnere) e il suo *stato interno* (acceso o spento). Una specifica lampadina rispetta il "*template*" della lampadina, ma ha sempre delle caratteristiche fissate; inoltre, possono esserci più istanze del concetto di lampadina, che potenzialmente avranno caratteristiche diverse ma possiamo svolgerci le stesse operazioni.

La *programmazione orientata agli oggetti* (OOP) è un paradigma di programmazione che va a generalizzare questa struttura, aggregando gli "*oggetti*" che andranno a modellare entità del mondo reale (o anche no). In questo modo, si rende più facile modellare altre entità separate e rendere il codice mantenibile.

X

2. Nozioni dell'OOP

#Definizione

Definizione 1 (nozioni base OOP).

In OOP definiamo le seguenti nozioni di base:

- Una *classe* è una rappresentazione di un concetto generico
- Ogni classe ha degli *attributi* (caratteristiche) e dei *metodi* (comportamenti)
- Un'istanza specifica di una classe si dice *oggetto*

In un certo senso, la classe fornisce un "*blueprint*" che ogni istanza della classe dovrà rispettare

Alcuni attributi e metodi di una classe potrebbero essere specifici dell'implementazione e vorremmo non renderli visibili dall'esterno. Da questo si hanno i meccanismi di *protezione dell'accesso*:

#Definizione

Definizione 2 (accesso OOP).

Ogni attributo/metodo di una classe può avere uno dei seguenti modificatori per regolarne l'accesso:

- *"Public"*: accessibile a tutti
- *"Protected"*: accessibile solo alla classe stessa e alle sue sottoclassi
- *"Private"*: accessibile solo alla classe stessa

Per decidere quale meccanismo di protezione usare, si può usare la seguente *"thumb of rule"*:

- Se dev'essere accessibile dall'esterno: pubblico
- Se è un dettaglio implementativo: protetto o privato

Però:

- Se rendiamo qualcosa pubblico, verrà sicuramente utilizzato e sarà difficile cambiarlo senza "rompere" dei codici che usano la classe
- Se rendiamo troppe funzionalità private, possiamo rendere una classe difficile da utilizzare

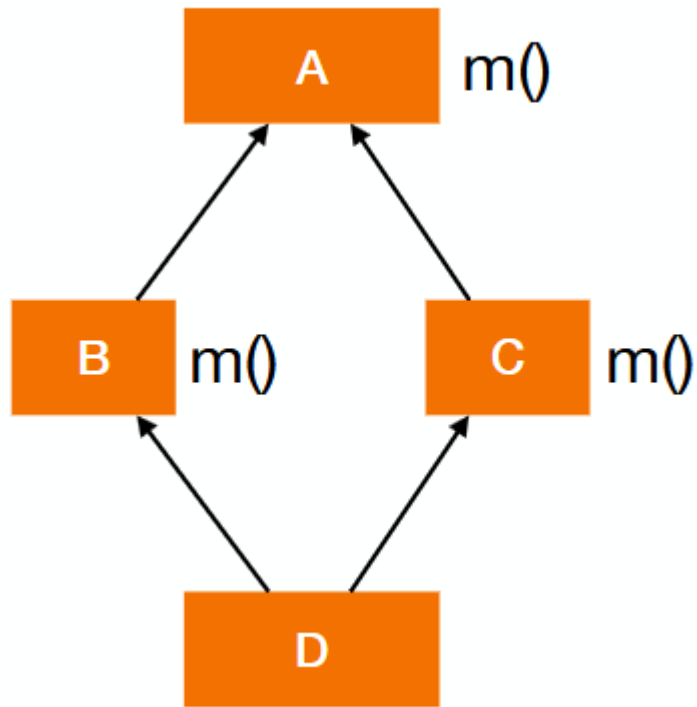
Siccome una serie di classi modellano concetti simili, potrebbe essere opportuno definire una *"gerarchia tra classi"* in cui abbiamo delle classi *"generiche"* con dei codici in comune da cui poi si può avere classi più specializzate che ereditano le parti comuni.

#Definizione

Definizione 3 (ereditarietà).

In OOP, si dice *superclasse* una classe comune che includerà il codice comune, da cui si hanno le *classi derivate*.

Osservazione. In certi casi è possibile ereditare da più classi base, e ciò potrebbe creare alcuni problemi. Ad esempio, illustriamo il *"diamond problem"*: una classe deriva da due classi che condividono la stessa superclasse. Da chi deriva il metodo?



Esiste un altro modo per mettere in relazioni classi diverse: la *composizione*

#Definizione

Definizione 4 (composizione).

La *composizione tra classi* consiste nell'incorporare oggetti di altre classi in una classe.

Nel design dell'OOP, si ha che:

- L'ereditarietà funziona meglio per le relazioni "*is-a*"
- La composizione funziona meglio per le relazioni "*has-a*"

OOP in Python

X

OOP in Python: definizione di classe, attributi di istanza e di classe. Convenzione per i meccanismi d'accesso. Metodi di classe. Dunder methods.

X

0. Voci correlate

- Programmazione Orientata agli Oggetti (OOP)

1. Sintassi OOP

Facciamo un ripasso veloce della sintassi per la programmazione OOP in Python.

Definizione Classe con Attributi e Metodi

```
class ClassName:
    self.a = ... # <-- attributo di classe
    def __init__(self, a1, ...): # <-- metodo
        self.a1 = a1 # <-- attributo di istanza
        ...

    def method(self, x): # <-- metodo
        ...
```

PYTHON

Osservazioni:

- L'argomento *self* indica l'oggetto stesso; in C sarebbe un puntatore alla struttura stessa
- Gli attributi definiti nel costruttore (`__init__`) sono *attributi di istanza*, invece quelli definiti fuori sono *attributi di classe*.
 - Se modifichiamo un attributo di classe, succede che questa verrà modificata per tutte le istanze!

Definizione Istanza

```
object = ClassName(a1, ...)
```

PYTHON

Accesso: In Python non c'è nessuna implementazione dei meccanismi di accesso alle funzionalità delle classi. Ci sono invece una serie di convenzioni per specificare il livello:

- *Public:* Nulla da fare

- *Protected*: Prima del nome bisogna aggiungere una sottolineatura `_`. Ad esempio `protected_attr` diventa `_protected_attr`; tuttavia, la funzionalità diventa comunque accessibile dall'esterno
- *Private*: Come prima ma con due sottolineature. Ovvero si ha `__private_attr`; in questo caso si effettua del *name mangling*, ossia dall'esterno viene visto come `_ClassName__private_attr`

X

2. Metodi di Classe e Metodi "dunder"

Metodi di Classe: I *metodi di classe* sono metodi della classe, che possono essere chiamati anche prima di creare oggetti. Quindi potrebbero essere utili per fornire metodi alternativi di costruzione di oggetti!

In Python la si definisce nella seguente maniera:

```
@classmethod
def classmethod(Class, ...): # Class è la classe, non un oggetto!
    cl = Class()
    ...
    return cl
```

PYTHON

Metodi Dunder: Alcuni metodi hanno un significato speciale:

- `__init__` è il costruttore dell'oggetto
- `__str__` fornisce una rappresentazione in stringa di un oggetto
- `__lt__`, `__le__`, ..., servono per definire un confronto tra oggetti
- `__getitem__(self, key)`, `__setitem__(self, key, value)`, `__delitem__(self, key)` serve per permettere l'accesso a dei valori usando una chiave, come se fosse un dizionario
- `__add__`, `__mul__`, ..., per definire operazioni tra oggetti (come somma, moltiplicazione, eccetera...)
- `__len__` per definire la "lunghezza" di un oggetto
- `__contains__(self, other)` per definire l'inclusione tra oggetti
- `__call__(self, ...)` per rendere un oggetto una *callable*, ovvero farlo comportare come una funzione (ad esempio, viene utilizzato in PyTorch!)

Esempio. Un esempio di applicazione dei *dunder methods* sono i *numeri duali*, in cui vogliamo definire la somma e il prodotto in una maniera specifica.

X

3. Ereditarietà

Per specificare che una classe eredita da una superclasse si usa la seguente sintassi:

```
class MyClass(SuperClass):  
    pass
```

PYTHON

In Python, ogni classe è implicitamente sottoclasse della classe `object`. Infatti, Python è proprio un linguaggio OOP.

In ogni classe derivata possiamo:

- Definire nuovi metodi e attributi
- Sovrascrivere metodi ("*overriding*")
- Accedere alla superclasse con `super()`

Osservazione. Python permette anche l'ereditarietà multipla, in tal caso basta specificare più "argomenti" di superclasse nella definizione della classe. Ovvero si scrive `class MyClass(S1, S2, ...)`. Nel caso di metodi condivisi tra superclassi, per capire l'ordine seguito per l'ordine con cui cerchiamo il metodo si usa `ClassName.mro()` ("*method resolution order*")

In Python si risolve la *diamond problem* dando la precedenza alla superclasse ereditata "*per prima*".

Copiare Oggetti in Python

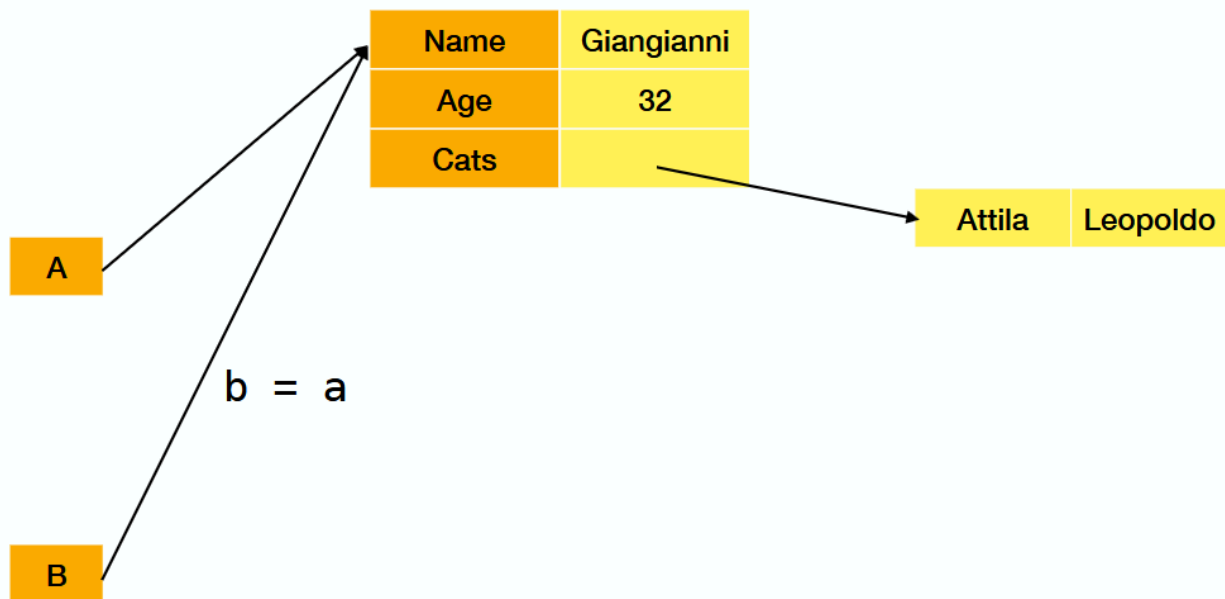
X

Copiare oggetti in Python: una breve osservazione su copie "shallow" e "deep".

X

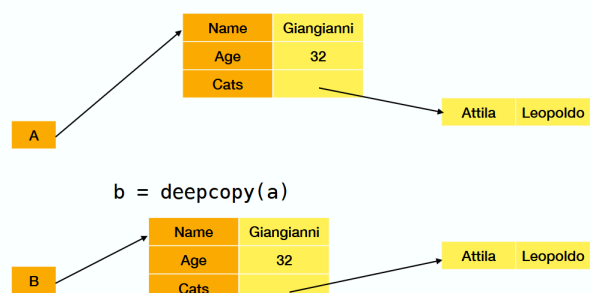
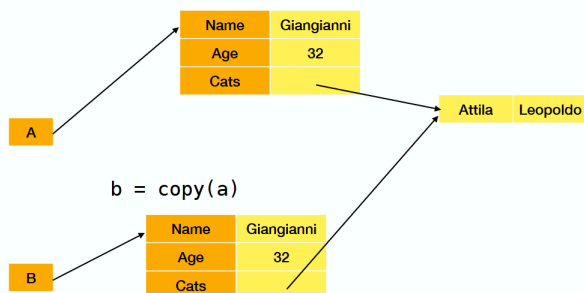
1. Copia di Oggetti

Osservazione. Quando facciamo un assegnamento, facciamo in realtà delle *assegnazioni ai riferimenti*, vale a dire che si lavora sempre sulle stesse istanze degli oggetti.



Tuttavia, in certi casi ciò non è quello che vogliamo. Per avere un metodo statico per costruire una *copia* a partire dall'originale esiste la libreria **copy** in Python, in cui si definiscono due metodi per effettuare delle copie:

- **copy** fornisce una copia "shallow" dell'oggetto, ossia copia tutti gli attributi in una nuova istanza dell'oggetto con degli assegnamenti; comunque gli attributi puntano agli stessi indirizzi di memoria
- **deepcopy** fa invece una copia anche di tutti gli attributi!



Se dovesse essere necessario fare delle *operazioni specifiche* per le operazioni di copia, si hanno le seguenti *dunder methods* per fare delle specifiche apposite:

- `__copy__(self)`
- `__deepcopy__(self)`

Eccezioni in Python

X

Eccezioni in Python

X

0. Voci correlate

- OOP in Python
- Programmazione Orientata agli Oggetti (OOP)

1. Eccezioni in Python

In *Python* possiamo usare le eccezioni per gestire gli errore.

Questi vengono "*sollevate*", da cui risaliamo lo stack delle chiamate fino a trovare quale parte del codice può gestirla. Quindi le eccezioni presentano due aspetti fondamentali: "*lanciare*" le eccezioni e "*catturarle*" per decidere come gestirle.

In Python si gestisce le eccezioni come segue:

```
try:
    ... # codice che può sollevare delle eccezioni
except ExceptionClass:
    ... # codice che gestisce le eccezioni
finally:
    ... # codice da seguire indipendentemente dal flusso delle eccezioni
```

PYTHON

Notiamo che in Python le eccezioni sono delle *classi*.

Invece per sollevare le eccezioni si usa `raise ExceptionClass()`. Per aggiungere informazioni aggiuntive, si ha il metodo `add_note(note)`.

Generatori in Python: motivazione, definizione di generatore come caso particolare di coroutine. Sintassi per definire generatori in Python: `yield` e `next`. Generator comprehension. Iterator protocol per definire generatori su oggetti. Libreria `itertools` per lavorare con iteratori (generatori).

0. Voci correlate

- OOP in Python

1. Concetto di Generatore e Coroutine

Idea. Iteriamo su oggetti senza dover caricarli completamente, generando un elemento alla volta. Potenzialmente si potrebbe iterare su collezioni infinite.

Questo è possibile grazie ai *generatori*, costrutto che ci permette di dire:

- Quale valore ritornare
- Come proseguire l'esecuzione dopo che il valore è stato ritornato

In Python si usano i *generatori*, un caso particolare di *coroutine*. I *coroutine* sono dei moduli con due comportamenti particolari:

1. Preservano lo stato delle variabili
2. Vengono sospesi in uscita, invece di essere completamente distrutti

In un certo senso, sono una specie di "*canale di comunicazione*". Per la definizione storica di *coroutine*, si veda la seguente reference:

 **Conway, Melvin E., *Design of a Separable Transition-diagram Compiler***

Under these conditions each module may be made into a *coroutine*; that is, it may be coded as an autonomous program which communicates with adjacent modules as if they were input or output subroutines. Thus, *coroutines* are subroutines all at the same level, each acting as if it were the master program when in fact there is no master program. There is no bound placed by this definition on the number of inputs and outputs a *coroutine* may have.

2. Generatori in Python

In python, ci sono due *keyword* principali per definire ed usare i generatori:

- `yield ...` per estendere un valore del generatore
- `next(...)` per chiamare il prossimo valore del generatore

Esempio. Il seguente generatore ritorna tutti i valori da 1 fino ad infinito:

```
def f():  
    k = 1  
    while True:  
        yield k  
        k += 1
```

PYTHON

Poi per usare il generatore si crea un'istanza della funzione e ci chiamiamo `next`.

```
x = f()  
  
next(x) # <- 1  
next(x) # <- 2  
...
```

PYTHON

Alternativamente, per creare un generatore si può usare le *generator comprehensions*, che funzionano in maniera analoga alle *list comprehensions*. La sintassi base è qualcosa del tipo `(... for ... in ...)`.

Esempio. `g = (x for x in range(10))` è un generatore che estende tutti i valori da 0 a 9

Per iterare invece su *oggetti*, si usa la convenzione *iterator protocol* per definire una maniera di iterare su oggetti. Bastano due metodi dunder:

- `__iter__(self)` per inizializzare l'iteratore
- `__next__(self)` per estendere i prossimi valori dell'iteratore

Definendo un oggetto con i due metodi dunder sopra, possiamo usarci i cicli for oppure la funzione `next` come se fosse un generatore.

Esempio. La seguente classe genera i cubi fino ad un numero massimo:

```
class Cubes:
    def __init__(self, nmax):
        self.nmax = nmax

    def __init__(self):
        self.n = n

    def __next__(self):
        if self.n > self.nmax:
            return StopIteration
        else:
            self.n += 1
            return (n-1)**3
```

Osservazione. Bisogna stare attenti al modo in cui designiamo le classi, soprattutto quando l'utente vuole effettuare delle modifiche sull'oggetto durante l'iterazione.

X

3. Libreria Itertools

Per lavorare su iteratori esiste la libreria *itertools*. Vediamo alcune delle sue funzionalità utili e comuni:

- `chain(a,b,...)`: Concatena più iterazioni
- `product(a,b,...)`: Considera il prodotto cartesiano di più iterazioni
- `pairwise(a)`: Considera una finestra temporale di dimensione due sull'array
- `permutations(a)`, `combinations(a,m)`: Considera le permutazioni e le combinazioni sull'array
- `repeat(x,m)`: Considera la ripetizione del valore x m volte. Se m non è specificato, allora $m = +\infty$.

L'implementazione di queste funzioni è semplice e si potrebbe anche rifare da capo usando i generatori, come ad esempio la concatenazione:

```
def mychain(*iters):
    for i in iters:
        for e in i:
            yield e
```

Per una documentazione dettagliata della libreria vedere il seguente link:

<https://docs.python.org/3/library/itertools.html>

Lavorare con Funzioni in Python

X

Lavorare con funzioni in Python: funzioni come "first-class objects". Funzioni che ritornano funzioni, concetto di closure di funzioni. Funzioni anonime e applicazioni parziali di funzioni. Mappatura, filtro e riduzione di collezioni con funzioni.

X

0. Voci correlate

- OOP in Python
- Generatori in Python

1. Funzioni di Funzioni

Le funzioni sono delle *first-class objects* in Python, ossia possiamo passare funzioni sia come argomenti di funzioni che come valori ritornati dalle funzioni.

Esempio. Considerare la seguente funzione:

```
def f():  
    def g(n):  
        return n  
    return g
```

PYTHON

Possiamo chiamare `phi = f()`, da cui `phi` è una funzione che si comporta proprio come `g`.

1.1. Closure

Q. Se volessi modificare `g` in `f` a seconda delle variabili esterne passate in `f`?

In questo caso si ha il concetto di *closure*, composta da due parti:

- Una funzione `f`
- Un ambiente che a ognuna delle variabili libere di `f` associa il valore che aveva al momento della definizione di `f`

Esempio. Considerare la seguente funzione:

```
def f(x):  
    def g(y):  
        return x+y  
    return g
```

PYTHON

In questo caso y è la variabile fissata, e x è la variabile libera catturata dall'ambiente.

Un caso più notevole di questo concetto è la definizione di funzioni parametrizzate, come la retta:

```
def r(m,q):  
    def val(x):  
        return m*x+q  
    return val
```

PYTHON

X

2. Funzioni Anonime e Parziali

Le *funzioni anonime* sono delle funzioni che non hanno un nome, ossia in Python non usufruiscono del costrutto `def`. Alternativamente, si usa la keyword `lambda` (lo stesso delle "*lambda functions*" del lambda-calculus di Alonso Church).

In Python le funzioni anonime sono limitate a definizione in singole espressioni.

Esempio. Si può definire la retta parametrizzata con le funzioni anonime:

```
def r(m,q):  
    return (lambda x: m*x+q)
```

PYTHON

Un altro caso utile sono le *applicazioni parziali di funzioni*, ossia data $f(x, y)$ e x fissata voglio definire l'applicazione $f_x(y)$. Questo è noto come l'operazione di *currying* ed in Python è possibile farlo in due modi:

- Usare le funzioni anonime
- Usare la libreria `functools` con la funzione `partial(f, args)`.

X

3. Map, Filter, Reduce

Vediamo altri modi per lavorare *con* le funzioni.

- `map(f, collection)`: Applichiamo la funzione f a tutta la collezione, senza dire nulla sull'ordine. Viene restituito un generatore su cui possiamo effettuare delle iterazioni
- `filter(f, collection)`: In questo caso si tratta di filtrare la collezione con f , che ritornerà un valore booleano
- `reduce(f, collection)`: Riduciamo tutta la collezione in un singolo elemento con f , che è un'operazione binaria. Ovvero da $[1, 2, 3, 4]$ otteniamo $f(f(f(1, 2), 3), 4)$.

Osservazione. Tutte le funzionalità sopra sono ottenibili usando le *list comprehension*

Decoratori in Python

X

Decoratori in Python. Idea dei decorator: sostituire f con $g(f)$. Esempio di decoratore. Notazione dei decorator in Python. Definire decorator parametrizzati.

X

0. Voci correlate

- Lavorare con Funzioni in Python

1. Idea dei Decoratori e Notazioni Preliminari

Idea. Siano f, g delle funzioni in Python, con g una funzione che accetta e ritorna funzioni. L'idea dei decorator è quello di sostituire f con $g(f)$

Esempio. Debugging, ad esempio per stampare gli argomenti della funzione e il valore di ritorno senza intervenire sulla funzione originale

Per affrontare i decorator, richiamiamo le seguenti notazioni in Python:

- `*args` per specificare una lista di argomenti o per appiattire una lista quando viene passata in un funzione
- `**kwargs` come `*args`, ma con dizionari

X

2. Decoratori in Python

Una struttura generale dei decorator in Python è come segue:

```
def decor(f):
    def wrap(*args, **kwargs):
        ...
        retval = f(*args, **kwargs)
        ...
    return retval
    return wrap # <--- the final "transformed" function

dec_f = decor(f)
```

PYTHON

L'unica parte "scomoda" di questo modo di definire il decoratore è il fatto che dobbiamo definire funzioni innestate. In Python esiste la seguente notazione:

PYTHON

```
@dec
def f(...):
    ...
    return ...
```

Ciò equivale a definire `dec_f = dec(f)`

Possiamo anche definire delle funzioni che ritornano dei *generatori*, come ad esempio

PYTHON

```
def dec_p(x):
    def dec(f):
        def g():
            ...
```

Con la notazione dei decorator abbiamo semplicemente

PYTHON

```
@dec(args)
def f(...):
    ...
    return ...
```

Che equivale a

PYTHON

```
decorator = dec(args)
@decorator
def f(...):
    ...
    return ...
```

Osservazione. Il decorator può anche tornare lo stato interno della funzione; inoltre, possiamo scrivere il decorator in un modo che la gestione della funzione sia totalmente generico!

Osservazione. Le funzioni hanno dei campi a cui possiamo accedere, come nome e doctring; quando li passiamo sotto un decorator, questi campi vengono "*mascherati*" dai decoratori. Per evitare questo si usa il decorator `@wraps(f)` all'interno della definizione del decorator per mantenere gli stessi nomi delle funzioni "decorate".

SEZIONE B. PYTHON E PRESTAZIONI

Libreria NumPy

X

Libreria NumPy in Python: motivazioni, gli array NumPy. Differenze tra array e liste. Tipi di dati in NumPy. Metodi per creare array, RNG in NumPy, usare gli array. Osservazione storica: linguaggi APL e K.

X

0. Voci correlate

- Copiare Oggetti in Python

1. Introduzione a NumPy

NumPy è una libreria per array ad alte prestazioni, usata come base per altre librerie (come Pandas e PyTorch). Inoltre questa libreria rende possibile il passaggio alla computazione su GPU.

Quindi, la libreria NumPy è molto importante per il calcolo numerico!

La struttura dati "centrale" di NumPy sono gli *array*. Sono:

- Diverse dalle *liste* Python in quanto queste sono invece più simili alle tabelle hash
- Simili agli array in C, con l'idea della *contiguità* e *compattezza* su cui applicare delle mapping "*pointwise*"

Quindi ci sono due assunzioni per usare gli array nella maniera più efficace possibile:

1. Assumere che la dimensione è più o meno costante (quindi statica), essendo che le operazioni di append/stack sono lenti
2. Considerare array con tipi di dato *omogenei*, anche se è possibile il caso eterogeneo. Infatti, ogni array ha un suo tipo di dato associato

In NumPy c'è una gerarchia di tipi: interi con e senza segno, e tipi floating point. Ad ogni tipo si specifica anche la sua precisione associata (8, 16, 32, 64 bit).

Oltre ai tipi di dati, gli array hanno altre proprietà:

- *Forma*: `array.shape` ritorna una tupla con le dimensioni dell'array
- *Dimensione*: `array.size` ritorna il numero di elementi contenuti nell'array, quindi una produttrice della sua forma
- *Casting*: `array.astype(...)` permette di fare il casting di array

2. Creare Array

Vediamo una serie di metodi per *creare* gli array

- `np.zeros(shape, dtype=...)`: Crea l'array con una forma specificata, riempiendo tutti gli elementi con zero
- `np.ones(shape, dtype=...)`: Stessa cosa ma con uno
- `np.full(shape, x, dtype=...)`: Stessa cosa di sopra ma con valore x arbitrariamente specificato
- `np.empty(shape)`: Ritorna l'array vuoto senza inizializzare le entrate. Notiamo che quindi se proviamo ad accederci i valori, non si può sapere cosa accaderebbe...
- `np.zeros_like(a)`, `np.empty_like(a)`, `np.full_like(a, x)`: Creare un array "copiando" la forma di un altro array, istanziando i valori come sopra (prima del "like")
- `np.arange(a,b,dx)`, `np.linspace(a,b,N)`: Creare un array con valori nell'intervallo $[a, b]$, o con uno step size costante dx o con N elementi totali

Alternativamente, possiamo creare degli array *casualmente*

Nota. Fino a poco tempo fa (e ancora oggi) ci sono delle funzioni apposite NumPy che generano casualmente i numeri. Si vede il metodo "moderno" per una serie di motivi:

- Funziona meglio per le applicazioni multi threading, in quanto il metodo di prima era sequenziale
- Si può imporre dei vincoli sul generatore
- Da un punto di vista crittografico è più sicuro

Per generare dei *numeri casuali* con NumPy, si inizializza prima il generatore con `rng = np.random.default_rng()` (o un altro generatore a scelta) e poi la si usa per generare i numeri, come ad esempio `rng.integers(a,b,n)`.

Vediamo il seguente elenco di metodi per creare dei numeri casuali (assumendo che `rng` sia il generatore):

- `rng.uniform(a,b,n)`: $\mathcal{U}^n([a, b])$
- `rng.random(n)`: Stesso di prima ma con $a = 0$ e $b = 1$
- `rng.standard_normal(n)`: $\mathcal{N}^n(0, 1)$
- `rng.normal(mu, sigma, n)`: $\mathcal{N}^n(\mu, \sigma)$

3. Usare Array

Vediamo adesso *come* usare gli array.

Il primo modo per usare gli array è quello di accedere degli elementi. Ci sono svariati modi per farlo:

- *Indicizzazione (con slicing)*: Possiamo usare un array NumPy come se fosse una lista Python per l'indicizzazione. Quindi possiamo fare `array[a:b:c]` per scegliere tutti gli elementi dalla posizione i -esima alla j -esima (esclusa), saltando ogni c elementi.
 - *Osservazione*. Con questo metodo si crea solamente una *vista* dell'array, ovvero non viene creata nessuna copia. Quindi se modifichiamo sulla parte indicizzata dell'array, modifichiamo anche l'array originale. Per evitare questo si usa il metodo `.copy()` (Copiare Oggetti in Python)
- *Fancy Indexing*: Alternativamente, si può usare una *lista* o una *maschera* di booleani per selezionare gli elementi di un array.
 - Per creare le maschere booleane possiamo scrivere espressioni del tipo `A>4`; poi per combinarle usiamo le operazioni binarie, tra cui `&`, `|`, `^`, `~`. Alternativamente, si può usare anche il metodo `np.where(...)`. Poi c'è anche l'equivalente dei quantificatori, con `np.any(...)` e `np.all(...)`
 - *Osservazione*. Qui facciamo una copia vera e propria dell'array
 - *Osservazione*. La sintassi per scrivere le maschere booleane è ispirata dal linguaggio APL, degli anni 60 ([APL](#))

Poi il secondo modo è quello di effettuare operazioni point-wise su array, ovvero data una funzione ad una variabile la applichiamo sull'array elemento per elemento. Vediamo una carrellata:

- `np.clip(a, min, max)`: Limitare i valori degli array, dal minimo al massimo specificato.
- `np.sqrt(a)`, `np.exp(a)`, `np.log(a)`, `np.sin(a)`, ...: Operazioni matematiche che ben conosciamo
- `np.dot(a,b)`: Prodotto scalare (o riga per colonna) tra due array (matrici). Esprimibile anche come `A@B`
- `np.sort(a)`: Ordinare l'array

Poi possiamo anche trovare *informazioni* sull'array, come ad esempio:

- `array.sum()`, `array.mean()`, `array.var()`, `array.std()`: Trovare la somma, media, varianza, deviazione standard dell'array
- `array.min()`, `array.max()`, `array.argmin()`, `array.argmax()`: Trovare i minimi, massimi e gli indici dei minimi e dei massimi

Libreria Multiprocessing

X

Parallelismo in Python: problematica del GIL, possibili soluzioni.

X

0. Voci correlate

- Programmazione Avanzata e Parallela
- Programmazione Multithreading con OpenMP
- Chiamare C da Python
- Mutex

1. Global Lock Interpreter

Problema. Python è un linguaggio *interpretato*, quindi per far eseguire del codice si usa l'*interprete* che traduce le espressioni Python in codice macchina. Tuttavia, per molti anni e ancora oggi si ha il *Global Lock Interpreter*, per solo un thread alla volta può accedere all'interprete di Python.

Questo rappresenta un problema nel realizzare la programmazione concorrente in Python. Tra le soluzioni abbiamo:

- Usare librerie "*GIL-free*", come ad esempio chiamare librerie C che usano OpenMP
- Eseguire Python in più processi, in questo modo ogni processo ha un suo interprete

X

2. Libreria Multiprocessing e Joblib

Un modo per eseguire Python con più processi è con la libreria standard *multiprocessing*.

La libreria *multiprocessing* permette di creare e controllare processi con una serie di costrutti ed oggetti definiti. Vediamo quelli più importanti

Process: Rappresenta un processo da avviare

- La si istanzia con `Process(target, args)` dove `target` è una funzione definita a priori
- Viene avviata col metodo `.start()` e terminata col metodo `.join()`
- Per conoscere l'identificativo del processo, si può usare la libreria standard `os` e chiamare `os.getpid()`

Queue: Rappresenta una coda condivisa tra più processi, su cui si può inviare e ricevere dati. In un certo senso è una specie di FIFO

- La si istanzia con `Queue()`
- Si inserisce e rimuove oggetti con `.put(...)` e `.get()`. Ovviamente entrambe i metodi sono potenzialmente bloccanti; un processo o aspetta che viene liberato, o ritorna un'eccezione da gestire

Lock: Rappresenta un mutex

- `.acquire()` per acquisire e `.release()` per liberare
- Ovviamente è possibile fare un deadlock

Value, Array: Rappresenta un valore o vettore da condividere tra processi

- La si istanzia con `Value(type, value)` o `Array(type, array)`

Pool: Rappresenta un insieme di processi a cui delegare una serie di lavori ("jobs")

- Istanziato con `Pool(n)`, con n il numero di processi
- Per delegare i lavori si chiama il metodo `pool.map(f, args)` con f la funzione target

Per altre funzionalità più comode, si usa la libreria *joblib* per programmazione multiprocessing.

Ad esempio, vediamo l'equivalente del PARALLEL FOR di OpenMP.

Parallel(n_jobs)(iterable): Una specie di parallel for

- Per applicare delle funzioni bisogna usare il metodo `delayed` della libreria, altrimenti viene già subito applicata (rendendo tutto inutile)
- Esempio: `Parallel(4)(delayed(f)(i) for i=1,...,100)` esegue $f(1), \dots, f(100)$ parallelamente con quattro processi. Notare che `(delayed(f)(i) for i=1,...,100)` è un generatore!

Libreria Numba

X

Compilazione *Just-In-Time*: libreria Numba per compilazione JIT in Python. Decoratori `@jit`, `@vectorize`, `@guvectorize`, `@jit(parallel)`, `@stencil`.

X

0. Voci correlate

- Programmazione Multithreading con OpenMP
- Chiamare C da Python

1. Compilazione Just-In-Time

La compilazione *just-in-time* (JIT) è una compilazione compiuta durante l'esecuzione del programma. Differisce dall'*interpretazione* di un programma, in quanto si spende del tempo per *compilare* un programma che può venire eseguito più volte.

Generalmente, è utile per del codice *numerico* e ripetuto più volte.

Vedremo come viene realizzata in Python.

X

2. Libreria Numba

Numba è una libreria Python che fornisce la funzionalità JIT tramite le LLVM. Lo fa mediante il decoratore `@jit`.

Esempio. Il seguente codice compila la somma dei quadrati degli interi:

```
@jit
def f(n):
    s = 0
    for i in range(0, n):
        s += (i**2)

    return s
```

PYTHON

Si dimostra che è *molto* più veloce del codice senza JIT. Tuttavia, non vale per il seguente codice:

```
@jit
def f_slow(l):
    s = ''
    for x in l:
        s += str(x)
    return len(s)
```

Infatti ci sono molte conversioni "inutili" nel mezzo, da cui in questo caso c'è perfino un *peggioramento* delle prestazioni.

Inoltre, possiamo specificare se parallelizzare la funzione compilata o meno con l'argomento `parallel`. Tuttavia, bisogna indicare *cosa* parallelizzare, usando alcuni costrutti specifici come `prange` al posto di `range`.

Esempio. Si parallelizza il prodotto scalare compilato:

```
@jit(parallel=True)
def jit_dot(u,v):
    s = 0
    for i in prange(0, u.shape[0]):
        s += (u[i]*v[i])
    return s
```

Esercizio. Scrivere il codice che faccia la matrice di moltiplicazione in JIT e in maniera parallela

Usare il solito algoritmo "naive" di fare la moltiplicazione riga per colonna, tuttavia parallelizzare solo gli ultimi due cicli esterni in quando nel ciclo più interno ci sono delle "race conditions".

soluzione

Poi ci sono altre funzioni utili in Numba, di cui vedremo alcuni.

`@vectorize`: Come in NumPy possiamo applicare operazioni su array in maniera point-wise, vogliamo generalizzare questa caratteristica su funzioni qualsiasi. Lo si fa con il decoratore `@vectorize`, che vettorizza la funzione in due modi:

- *"Eager"*: Se si specifica la signature della funzione passandolo in argomento del decoratore (vedremo come), la funzione viene compilata subito
- *"Lazy"*: Altrimenti se non si specifica la signature, viene compilata solo al momento della chiamata della funzione

Esempio. Vettorizzare l'operazione "fma", che fa la moltiplicazione tra due valori e la somma per un terzo valore

PYTHON

```
@vectorize([float64(float64, float64, float64)])
def vectorized_fma(a,b,c):
    return a*b + c
```

Inoltre possiamo specificare anche il "target" in cui compilare il codice, aggiungendo l'argomento `target` al decoratore. Possiamo impostarci tre valori:

- `target="cpu"`: Opzione di default, esegue sul CPU in single thread
- `target="parallel"`: Parallelizzazione multi-thread su CPU
- `target="cuda"`: Parallelizzazione su GPU usando CUDA (molto complicato!)

`@guvectorize`: Invece se si hanno argomenti diversi nell'operazione vettoriale (come ad esempio due vettori ed uno scalare), allora bisogna usare la funzionalità più generica `@guvectorize`. Questo non ha un valore di ritorno, e l'array da riempire dev'essere passato come argomento; inoltre, qui è obbligatorio specificare la signature della funzione

Esempio. "fma" ma con termine di somma costante

PYTHON

```
@guvectorize([float64[:], float64[:], float64, float64[:]], "(n),(n),()->(n)")
def fma_fixed_c(a,b,c,out):
    for i = 0, ..., a.shape[0]-1:
        out[i] = a[i]*b[i] + c
```

Osservazione. Non c'è nessun valore di ritorno, che non viene specificato nella signature dei "tipi di dati"; viene invece specificato nella signature a destra.

Vediamo adesso l'ultima funzionalità, una abbastanza particolare ma utile: l'operazione "stencil".

`@stencil`: Un pattern abbastanza "comune" nel contesto delle simulazioni è quello dell'operatore che agisce su "kernel", ovvero abbiamo un "formino" che viene applicato su regioni locali di una matrice. In questo modo, l'utente può definire lo "stencil" a parte con Numba e poi applicarlo in un altro codice come se fosse un'operazione che agisce su array.

Esempio. Il seguente kernel fa la media di un nodo con i suoi primi vicini (su una griglia):

PYTHON

```
@stencil
def stencil_2D(k):
    return (k[-1, -1]+k[-1, 0]+k[-1, 1]+k[0, -1]+k[0, 0]+k[0, 1]+k[1, -1]+k[1, 0]+k[1, 1])/5
```

Possiamo generalizzarlo usando i cicli for, tuttavia bisogna passare l'argomento `neighborhood=((a1, b1), (a2, b2))`. Una riformulazione del codice precedente è come segue:

PYTHON

```
@stencil(neighborhood=((-1, 1), (-1, 1)))
def stencil_2D(k):
    r = 0
    for i in range(-1, 1+1):
        for j in range(-1, 1+1):
            r += k[i,j]
    return r/5
```

Poi la seguente funzione compilata "just-in-time" la applica su un array:

PYTHON

```
@jit
def apply_stencil(A):
    r = np.zeros_like(A)
    stencil_2D(A, out=r)
    return r
```

Documentazione sullo stencil: <https://numba.readthedocs.io/en/stable/user/stencil.html>